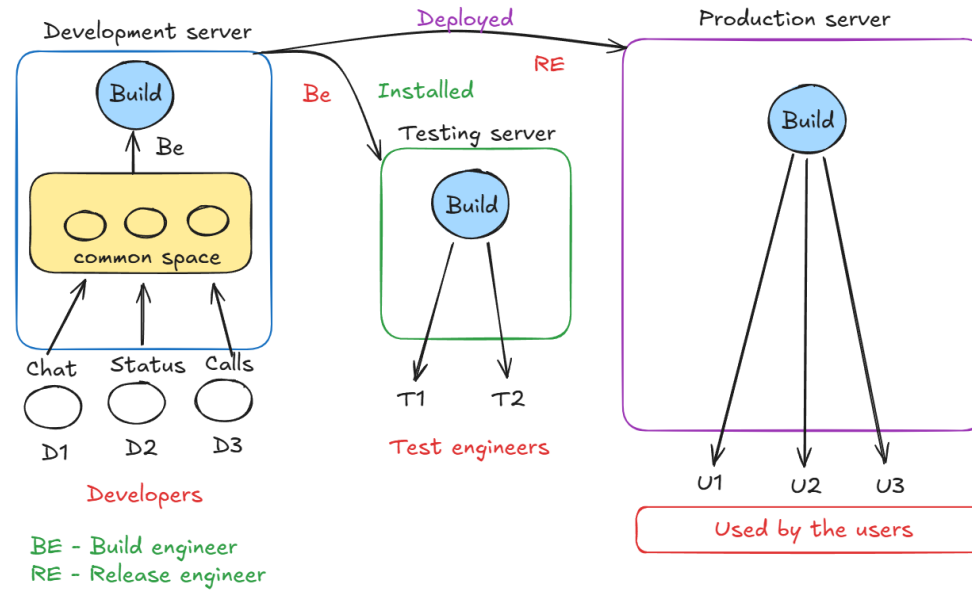


Jenkins

1. Traditional approach & release process
2. Introduction to jenkins
 - what is jenkins
 - what is CI/CD?
 - history of jenkins
 - features of jenkins
 - alternative tools of jenkins
3. Dashboard explanation after installation & setup of jenkins
4. Job & its types
 - what are jobs?
 - what is freestyle job
 - what is maven project
 - pipeline job & its types
5. freestyle - running java file and maven project from github in AWS
6. Installation & setup of jenkins in windows
7. freestyle - running java file and maven project from github in windows
8. freestyle - running spring petclinic project from github in windows
9. Maven job - running maven project & spring petclinic application from github using maven pluggin
10. Build triggers
 - build periodically
 - poll scm
 - github webhook
11. Role based authorization
12. Pipeline & its types
 - declarative pipeline
 - Scripted pipeline
13. CI/CD implementation
14. Master-slave architecture.

Traditional approach & release process



Drawbacks

1. It is time consuming
2. It involves manual workload
3. Higher cost
4. Less efficiency
5. Slow development life cycle
6. Human errors.

To overcome all the drawbacks we have to go for JENKINS

Introduction to jenkins

What is jenkins ?

- > It is the free and open source CI/CD tool that is written by java which automates the process of building, testing & deploying the software automatically
- > It is helpful to developers & DevOps team to implement the CI/CD process

What do we need jenkins ?

Without Jenkins, developers have to do the work starting from building to deploying manually that leads to time consumption and human errors.

With Jenkins, every manual step can be automated → Developers, after writing the source code, can just commit it and the remaining work will be completely done by Jenkins.

Jenkins will pull the source code, build, test & deploy the application.

What is CI/CD ?

Continuous integration

It will continuously collect the source code from remote repository(github) and generate the build.

Continuous delivery

It is the process of generating the build for testing before it gets installed/deployed in production server.

Continuous deployment

It is the process of continuously installing/deploying the stable build into the production/client server automatically.

History of jenkins

2004 - Kohsuke kawaguchi, he created the project called as 'hudson' to automate the process of building & testing the application. He worked in sun microsystem.

2010 - When oracle acquired sun microsystem it created the conflict between hudson & oracle.

2011 - Hudson project was renamed as "JENKINS"

Features of jenkins

1. It is free & open source
2. It supports CI/CD
3. It automates the task
4. It is very user friendly
5. It is scalable too
6. It is extensible with pluggins
7. It works on any platform - windows, linux, macOS.....
8. It supports master slave architecture
9. It supports pipeline as a code
10. Community support & easy installation & configuration.

Alternative tools of jenkins

1. Teamcity (Jetbrains)
2. Github actions
3. Circle CI
4. Travis CI
5. Azure pipelines
6. AWS codepipeline
7. Bamboo (Atlassian)
8. Gitlab CI/CD

Jobs

A job in Jenkins is basically a task or project that Jenkins runs.

-or-

A job is a specific task which is performed in Jenkins and through a job we can build code, run tests and deploy applications.

Types of job in Jenkins

1. Freestyle job
2. Maven job
3. Pipeline

1. Freestyle job

In Jenkins it is the most basic type of job.

It allows you to define simple tasks such as building code, running tests and deploying the software.

Here, it is not required to install any additional tools as it is pre-installed.

It has point-&-click setup that means no coding is required.

It also provides an easy-to-use GUI.

Features of freestyle job

1. Easy to create and configure
2. It can pull code from Git, SVN or other repositories as well
3. It can run build tools (Maven, Gradle, Ant)
4. It can also execute Bash script/Shell script
5. Very much suitable for beginner projects or small projects.

2. Maven job

It is a job that is specifically designed for projects that use Maven.

It is designed to build and manage Java projects using Maven that means it automates the process of building, testing the Java projects.

Instead of manually writing the build steps, Jenkins will read Maven's pom.xml file and then run everything automatically.

Features of maven job

1. It is designed for java projects
2. Understands pom.xml
3. No manual build steps needed
4. It is good in automatic dependency handling
5. Easier for continuous integration.
6. Supports build lifecycle.

3. Pipeline job

In jenkins it is a job where you write the steps of your build, test & deploy the process as a code.

We need write down the pipeline scripts with groovy language.

This code is usually written in a file called as Jenkinsfile.

It is more powerful and flexible than a freestyle job.

Features of pipeline job

1. Pipeline as a code
2. Supports complex workflows
3. Helps in error handling
4. It is reusable
5. It is version control friendly

steps to perform java project in freestyle

1. Login for jenkins & click on new item or click on create job
2. Give the job name & select the job type --> freestyle
3. Give the description as it is optional
4. Under source code management click on git
5. Paste the repository link of java project from the github
6. Under build step select --> execute windows batch command
7. Give the command to compile and run the java project

Command : To compile --> `javac <filename>`

To execute --> `java <filename>`

8. Click on apply and save
9. Click on build now.
10. To check output go for console output.

Steps to run a maven project in a freestyle job

1. Log in to jenkins and click new item (or create job).
2. Provide a project name and choose the job type as maven project.
3. Optionally, add a description for the job.
4. Under source code management, select git & enter the github repository url of your java project.
5. In the build steps section, choose execute windows batch command.
6. Add maven commands to generate the package and site : `mvn package site`
7. Save the job by clicking apply, save & start the build by clicking build now.
8. Finally, open the console output to verify the results.

Workspace

>> It is the folder or directory where jenkins will be storing all the files which is related to the job during its execution.

It includes :

1. source code
2. dependencies
3. test results
4. compiled files

What does workspace consist of ?

1. It will download the source code from the repository(github, gitlab) into the workspace.
2. It will be storing the build scripts as well(In maven it is pom.xml)
3. It compiles the files(.class from .java)
4. It will save reports & files that are temporary(test results, logs, code coverage reports)

Why workspace is important?

1. Each job has its own workspace so that the jobs does not clash with each other therefore we have workspace .
2. If anything needs to be debugged, then first you need to go inside the workspace directory & verify what files the jenkins have used.
3. After the job is completed, you still can check the workspace to see what jenkins has used.

Steps to run sample maven project using maven job

1. Install maven integration plugin
 - Login to jenkins.
 - From the jenkins dashboard go to manage jenkins.
 - Open manage plugins and search for "maven integration" plugin, then install it.
 - Check if the plugin is successfully added under installed plugins.
2. Configure maven path in tools
 - From the jenkins dashboard go to manage jenkins.
 - Click on tools & select maven and then click on add maven.
 - Provide a name (Ex : Maven-home) & mention the installation path of maven installed in your system.
 - Click on apply and then save the settings.
2. Create a job using maven project
 - Go to jenkins dashboard and click on new item.
 - Enter a job name & choose maven project as the job type.
 - Click on next & fill in the job description if needed.
 - Under scm, select git
 - Provide the repository url of the maven sample project.
 - In the build goals section specify the maven goals (for example: clean package site).
 - Click on apply and then save.
 - Finally, click on build now to run the job.

Steps to run Spring Petclinic project using maven job

1. Create a job using maven project
 - Go to jenkins dashboard and click on new item.
 - Enter a job name
 - Choose maven project as the job type.
 - Click on next & fill in the job description if needed.
 - Under scm, select git & provide the repository url of the spring petclinic project.
 - In the build goals section specify the maven goals (for example: package).
 - Click on apply and then save.
 - Finally, click on build now to run the job.

Steps to setup jenkins in AWS instance

1. Launch EC2 instance with the Ubuntu operating system specifying t2.medium as the instance type.
2. Establish a connection to the newly launched instance to proceed with the installation of Java, Maven and Jenkins
3. Before installing Jenkins, you need to set up its dependencies. Execute the following commands :

`sudo apt update` --> to update the package manager.

`sudo apt install openjdk-17-jdk -y`

`sudo apt install maven -y`

Steps to Install and Start Jenkins

1. Navigate to the official jenkins webpage by searching "download jenkins in linux".
2. Locate and copy the installation script designated for debian/ubuntu that corresponds to the "long term support release".
3. Paste this script into your EC2 instance's terminal & execute the script that will install Jenkins then enable and start the jenkins service.

`sudo systemctl start jenkins` --> to start jenkins service.

`sudo systemctl status jenkins` --> to check the current status of jenkins.

Accessing the Jenkins Web Interface

1. Configure your EC2 instance's security groups to enable port number "8080" (this is the default port for jenkins).
2. Copy the public IP address of your instance and paste it into your web browser followed by the port number (`http://<public-IP>:8080`) to access the jenkins login page.

Initial Jenkins Setup

1. To complete the setup, you'll need the initial administrative password so copy the file path where this password is stored and use the `cat` command within your EC2 instance to display its contents : `sudo cat <administrative-path>`
2. Paste the administrative password into the setup wizard, click next and then provide the required credentials to finalize the setup and begin using jenkins.

Note : Follow same steps to run java, maven & spring petclinic project using both freestyle & maven job type.

- When configuring a build step in linux, select "execute shell" for the build command.

Build trigger

>> Build trigger will tell the jenkins to start the job automatically instead of human doing it manually.

Why build trigger is needed?

1. To avoid manual steps.
2. It makes the CI/CD process automatic.
3. Fast response - here, developers will get the result quickly after they push the code.
4. It helps in scheduling the build generation.

Build trigger types

1. Manual build generation - by clicking on "build now" manually will help in generating the build
2. Build periodically - will be using cron job to schedule the build generation.
3. SCM polling - will check for changes in the github repo and if it finds the changes, it will generate the build
4. Github webhook - Github will call jenkins as soon as the code has been pushed & it is more efficient as well.
5. Build after other project are built.

Build periodically



setting the time

- >> You can make a job get executed automatically in given time without clicking on "build now".
- >> To implement the build generation automatically on specified time we will go for the "cron expressions"

Cron expression fields

1. minute
2. hour
3. day of month
4. month
5. day

Syntax :

minutes	hour	day_of_month	month	day
(0-59)	(0-23)	(1-31)	(1-12)	(0-7)

↓
(0--> sunday
7--> sunday)

Ex: 0 0 * * *

Note : * $\rightarrow$ every

0 $\rightarrow$ min

0 --> hour --> at midnight

* --> every day of month

* --> every month

* --> every day

It will trigger the build every day at midnight 12.

hour - 12:00

↓
(minutes)

hour - 00:00 --> 12am

↓
(minutes)

0/5 * * * *

0/5 --> every 5 minutes

* --> every hour

- * --> every day of month

- * --> every month

* --> every day

It will trigger the build every day after every 5 minutes.

0 */2 * * *

0 --> at 0 minute

* / 2 --> every 2 hours

- * --> every day of month

- * --> every month

* --> every day

O/P : 12:00

2:00

4:00

6:00

It will trigger the build after every 2 hours

* */2 * * *

2 */2 * * *

O/P : 12:00

2:02

4:03

6:04

8:05

It will trigger the build after every 2 hour and 2 minutes

45 17 11 7 *

It will trigger the build at 5:45 on 11th july.

Note : > As per the type that generates the build on given interval, irrespective of the result it will still follow the schedule may it be success or failure.

> It will trigger & will start generating the next build on given time even if the previous build fails.

Steps to implement build periodically

1. create job in jenkins and name it specifically using freestyle job.
2. Give description and add the github url of the project(java/maven/spring petclinic/any project)
3. Add branch name as per the project.
4. Under the build triggers section enable the build periodically option and provide a cron expression in the schedule field to define how frequently the new build should get generated (Ex : * * * * * --> Every 1 min build will get generated).
5. Under build step section add "execute windows batch command" to execute the required commands during the build generation process.
6. Click on apply and save & then the build will get generated based on the time interval without clicking on "build now" option.

2. Poll SCM (source code management)

Jenkins checks the github repository at the specified time interval to see if there are any changes done.

Changes done --> modifying the source code
new commits
removing the lines from the code
adding the lines in the code.

As soon as jenkins finds the changes in github repository it will trigger the build and if there are no changes found then jenkins will do nothing.

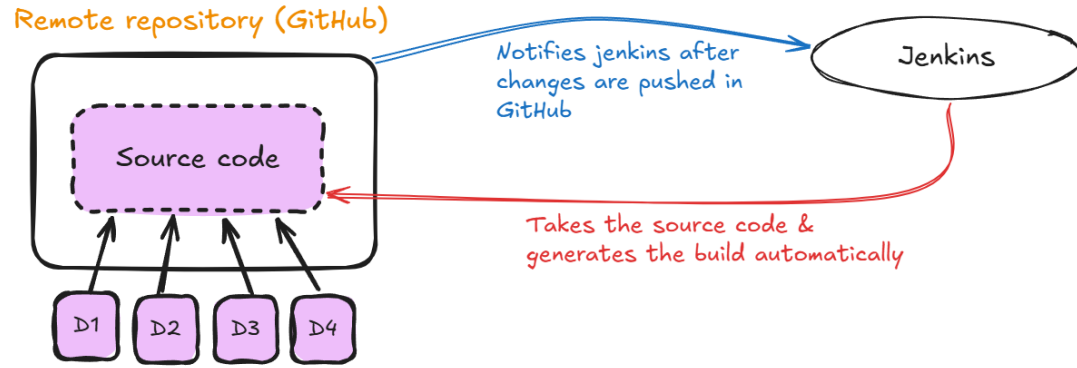
Steps to implement Poll SCM

1. create job in jenkins and name it specifically using freestyle job.
2. Give description and add the github url of the project(java/maven/spring petclinic/any project)
3. Add branch name as per the project.
4. Under the build triggers section enable the Poll SCM option and provide a cron expression in the schedule field to define how frequently jenkins should check the remote repository for changes(Ex : */2 * * * * --> Every 2 mins jenkins will check remote repository for changes)
5. Under build step section add "execute windows batch command" to execute the required commands during the build generation process.
6. Click on apply and save & the first build will be generated at first check based on time interval.
7. Later to generate the 2nd build we should make changes in the source code and commit it or push the source code in remote repository.

Note : In Jenkins the Poll SCM trigger only starts a new build when it detects changes in the remote repository.
In case if no new commits are found since the last build jenkins will skip triggering a new build.

3. GitHub hook trigger for GITScm polling

- > It is the build trigger in Jenkins where GitHub itself will notify Jenkins whenever there are code changes in GitHub so that the Jenkins can immediately generate a new build without waiting for scheduled time intervals.
- > Here, cron expressions are not provided as GitHub itself will notify Jenkins about changes in source code in remote repository.
- > Therefore it is event based and not time based.



Benefits : Efficient & faster process, automated process, good collaboration, immediate trigger.

Steps to implement GitHub webhook

1. Go to your GitHub repository.
2. Click on settings then webhooks and click on add webhook.
3. In the payload URL field enter your Jenkins URL followed by "/github-webhook/".

Example:

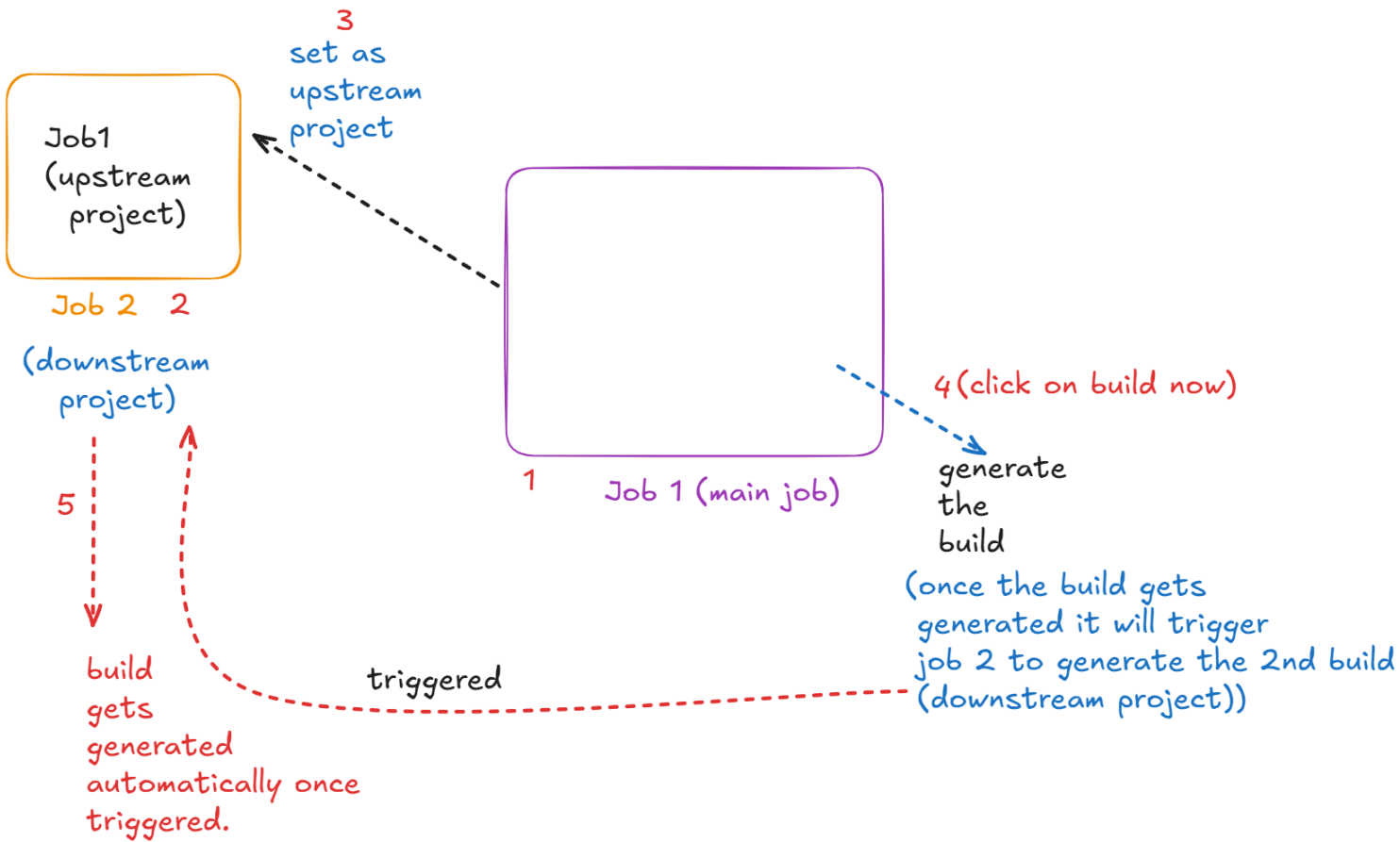
<http://24.123.214.200:8080/github-webhook/>

4. Under content type select application/json.
5. In the events section choose "send me everything" or "push event" option.
6. Click add webhook to save the configuration.

To verify

- >> Make a change in the remote repository
 - commit changes in GitHub
 - push the changes made through git bash.
- >> Go to Jenkins to check whether a new build is automatically generated or not.

4. Build after other projects are built



Steps to implement build after other projects are built

1. create 2 jobs in jenkins and name them specifically.
2. In second job set up job 1 as upstream job by selecting :
 - build after other project are built
3. Add the project which needs to be generated first(upstream project) in build triggers
4. configure rest of the configurations of that job then apply & save
5. Go to job 1(upstream project) and click on build now to generate the build.
6. Once after build gets generated check the job 2(downstream project) whether it is triggered or not. As it gets triggered it will generate the build(downstream project).

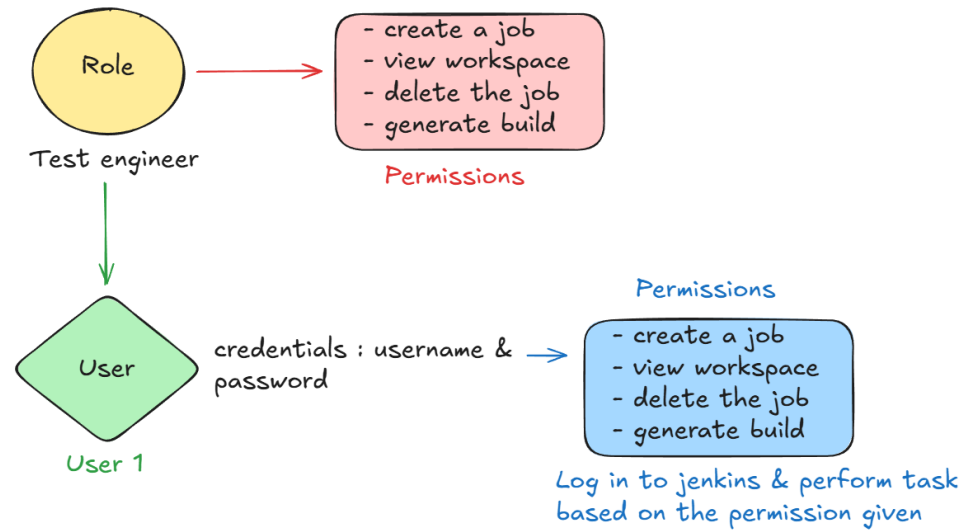
Role based authorization

Role based authorization is giving permission to the user based on their role and not individually.

Role : A set of permissions - what actions are allowed to do.

User : A user is the one who will log in to jenkins and perform certain actions based on the permission given to that user.

For Ex :



Note : We cannot use role based authorization in jenkins.

Therefore to implement role based authorization we have to install RBA strategy plugin.

Why is role based authorization required ?

- To control who can perform specific tasks/actions.
- Not everyone can have admin access so it helps to give exact permission to right users in order to keep the system organised and secure.

Features of RBA

1. Security
2. Systematic Management
3. Scalability & flexibility
4. Reduced human errors.
5. Team collaboration

To implement role based authorization

1. Install role based authorization plugin
 - Open jenkins dashboard then click on manage jenkins and then select plugins --> available plugins.
 - Search for "role based authorization strategy" & install the plugin and verify in installed plugin by searching "role based authorization strategy".
2. Configure security settings
 - Go to manage jenkins and select security & under security realm select "jenkins' own user database".
 - Under authorization select "role based strategy" and click on apply and save.
3. Manage roles and set permissions
 - After enabling role based strategy go back to manage jenkins and under security new option will appear that is manage and assign roles.
 - Click manage roles & under global roles create a new role (Ex : Test engineer, Developer, Manager etc)
 - Assign specific permissions (Ex : In overall : read permission, In job : create, configure, workspace, build, read) and click apply and save.
4. Create a new user
 - Go to manage jenkins and then select users and click on create user.
 - Provide the following details such as username, password, confirm password, full name, email address & click on create user.
5. Assign role to user
 - Go to manage jenkins and click on assign roles & locate the newly created user.
 - Check the box corresponding to the role (Ex : test engineer) you want to assign then click on apply and save.
6. Verify role based access
 - Log out from the admin account & log in with the newly created user's credentials to verify that the user can perform only the actions permitted under the assigned role (Ex : create, configure jobs, generate build etc)
7. Modify role permissions (If needed)
 - Log back in as the admin and then go to manage roles and modify the permissions for the existing role as required.
 - Click apply and save & log in again as the newly created user to verify that the updated permissions are reflected or not.

Pipeline & it's types

Pipeline

- >> It is the scripted instructions that we need to follow in order to perform the tasks.
- >> It tells jenkins how to build, run tests and deploy the project step by step.

Why is pipeline needed?

1. To automate the process
2. It saves time
3. consistent & scalable
4. No repetitive task
5. It is reusable

Types of pipeline

1. Declarative pipeline
2. Scripted pipeline

1. Declarative pipeline

- >> It is a basic and structured way of writing the pipeline that has the fixed structure that is predefined format.
- >> It has human friendly syntax that is why it is easy for beginners to understand.

Why we have go for declarative pipeline?

1. Easy to understand
2. Structured
3. Consistent
4. user friendly syntax

A declarative pipeline includes :

1. Pipeline
2. Agents
3. Stages
4. Stage
5. Steps

Syntax :

```
Pipeline {  
    agent any  
    stages {  
        stage ('<stage-name>')  
        {  
            steps {  
                #Commands to perform task  
            }  
        }  
    }  
}
```

Steps to implement a declarative pipeline for the project in jenkins

1. In jenkins create new job using pipeline option as the job type and click ok.
2. In configuration page provide description & under source code management section enter git project repository url copied from github followed by branch name.
3. Go to pipeline section and select the sample script hello world from dropdown menu.
4. Modify the sample script as follows:
 - Replace the default stage name "hello" with "git clone".
 - Go to pipeline syntax & select "git: Git" from the sample step options then provide the repository url and click on generate pipeline script.
 - Copy the generated script and paste it inside the "steps" block of the "git clone" stage.
5. Duplicate the stage block and paste it below to create a new stage for java compilation.
 - Replace the stage name with "Java-project".
 - Go to pipeline syntax select "bat: windows batch script" from the sample step options then provide the commands and click on generate pipeline script.

```
javac test.java  
java test
```
 - Copy the generated script and paste it inside the "steps" block of the "Java-project" stage.
6. Click on apply and then save to store the configuration.
7. Click on build now to execute the declarative pipeline and verify the build output.

Note 1: Follow the same steps to run a maven sample project using the pipeline job type.

- When configuring select bat: windows batch script then provide the command `mvn package` and click on generate pipeline script.

Note 2: Follow the same steps to run a java & maven sample project using the pipeline job type in ubuntu.

- When configuring in ubuntu select sh: shell script then provide the required commands and click on generate pipeline script.

2. Scripted pipeline

It is the pipeline that defines step by step instructions for how your application should be built, tested & deployed. It is written in groovy scripting that allows you to write code to control how jenkins builds, run tests, and deploy the application.

What is groovy scripting?

It is a scripting language to write pipeline script & it uses java programming language. It offers features such as conditional statements, variables, looping statements and so on.

Advantages of groovy scripting

1. You don't have to declare the type or specify the type.
2. Useful for automation purpose & it is flexible.
3. As it is java compatible so we can use java libraries.

Why do we go for scripted pipeline?

1. Since we are using looping statements, variables, functions so it is more powerful.
2. You can reuse the script.
3. It is more useful when it comes to complex CI/CD pipelines.

Syntax :

```
node {  
    stage('<stage_name>')  
    {  
        #define task or statements or commands  
    }  
    .....  
}
```

Sample example of groovy scripting

```
node {  
    def url = 'https://github.com/TANVIP-6M/mavenf.git'  
    def stagelist = ['compile','test','package']  
  
    stage('git-clone') {  
        git url  
    }  
    for (i in stagelist)  
    {  
        stage(i)  
        {  
            if(i == 'compile')  
            {  
                bat 'mvn compile'  
            }  
            if(i == 'test')  
            {  
                bat 'mvn test'  
            }  
            if(i == 'package')  
            {  
                bat 'mvn package'  
            }  
        }  
    }  
}
```

Email setup to get notifications from jenkins

1. Install a plugin in jenkins called "email extension template plugin".
2. Go to your gmail account, enable 2-step verification, and under settings, search for app password.
3. Create a new app password with the name jenkins and click create.
4. Copy the generated password and click ok.

Open jenkins and follow these steps :

1. Go to manage jenkins & then click on system.
2. Under email notifications in the smtp server field enter smtp.gmail.com.
3. Enable the use smtp authentication option.
4. Enter your gmail address under username, and paste the app password you copied under password.
5. Check the box for use ssl & set the smtp port to 465.
6. Test the configuration by sending test email to confirm it works then click apply & save and check your inbox for the test email from jenkins.

Script to get notifications through pipeline

```
post
{
    success
    {
        mail (
            bcc: "",
            body: "Hello this is to inform
                  that the build got
                  generated successfully",
            cc: "",
            from: "",
            replyTo: "",
            subject: 'java build success
                    notification',
            to: 'abc@gmail.com')
    }
}
```

Example :

```
pipeline {
  agent any

  stages {
    stage('Git-clone') {
      steps {
        git 'https://github.com/<github_acc_name>/java.git'
      }
    }
    stage('java_project') {
      steps {
        sh '''javac Test.java
              java Test.java'''
      }
    }
  }
  post
  {
    success
    {
      mail (
        bcc: "",
        body: "Hello
              this is to inform that the build got generated successfully",
        cc: "",
        from: "",
        replyTo: "",
        subject: 'java build success notification',
        to: 'abc@gmail.com')
    }
  }
}
```

Note : In the pipeline syntax section, choose "mail".

- If you want to receive an email when the build fails instead of when it succeeds replace post "success" with "failure" in the pipeline script.

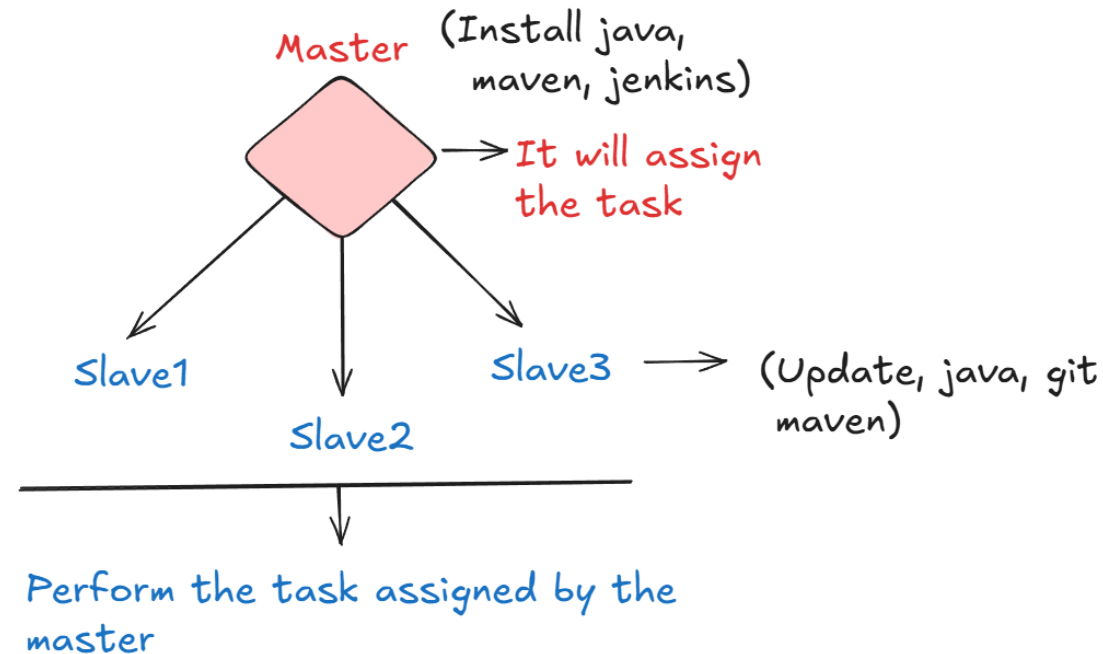
Master Slave Architecture

The master slave architecture is a term/model in Jenkins which consists of main and secondary systems.

Here, master will assign the work/task and slaves are going to perform the assigned task.

Why is it needed?

1. Reduce the workload
2. Improvement in performance.
3. Reliability
4. Systematic management



Slave is also called as node and agent(alternative names)

Execution steps to create jenkins master and slaves with different job types

Step 1: Create five instances

- Instance 1 --> jenkins (enable 8080 port number)
- Instance 2 --> slave1 (for freestyle job)
- Instance 3 --> slave2 (for maven job)
- Instance 4 --> slave3 (for declarative pipeline job)
- Instance 5 --> slave4 (for scripted pipeline job)

Step 2: Connect to jenkins instance & slave instances

- Connect to jenkins instance then run the commands to update, install java, maven & jenkins (start & check status)
- Connect to slave1, slave2, slave3, slave4 instance & run the commands to update, install java in all slave instances.
- Create directory in every slave instances (slave1 - f1, slave2 - f2, slave3 - f3, slave4 - f4)

Step 3: Sign in to jenkins

- Copy the public ip of the master instance and open it in the browser using port 8080 (Ex: <http://<public-ip>:8080>).
- Unlock jenkins using the initial admin password (sudo cat <path of admin password>) and complete the setup by installing suggested plugins.
- Install maven integration plugin & under maven installations click add maven give the name as "MAVEN_HOME" and provide the maven path (from "/usr/share/maven" or use "mvn --version" command to check path) and save the configuration.

Step 4: Configure jenkins nodes for every slave instances

- In jenkins dashboard go to manage jenkins and select nodes then click on new node.
- Give the node name as "slave1" & select permanent agent then click create and provide description if needed (optional).
- Set number of executors to 2 (or any number) & for remote root directory go to slave instance then get inside the folder (cd f1) & run pwd then copy that path and paste it in remote root directory field.
- Provide label as s1 (or any name) & set usage as "use this node as much as possible".
- In launch method select launch agent via ssh & in host enter slave1 instance public ip.
- Under credentials click add then click jenkins & select global credentials (unrestricted) & in kind select ssh username with private key.
- In scope select global jenkins and under id enter 01 (or any number) & in description enter "slave1_instance".
- In username type ubuntu & in private key select enter directly and paste the content of the key pair created while launching slave instances.
- Click add & select created credential from the credentials field & in host key verification strategy select non verifying verification strategy.
- In availability select keep this agent on as much as possible & click save.
- Click on the created slave node and check if it is online or not therefore click on log to ensure that the agent is successfully connected & online.

Step 5: Create job (sample java project) using freestyle job for slave1

- Go to new item, give the job name, select freestyle job and click ok.
- Provide description, enable the checkbox of restrict where this project can be run and enter the label "slave1".
- Under source code management, select git & give repository url of the java project.
- Specify the branch_name, add build step --> execute shell & add required commands to generate pipeline script, apply & save then click on build now.
- Verify in slave1 instance by navigating to "f1" directory and run "ls" & you should find "workspace" then get inside workspace and verify your project files are present.

Step 6: Create job (sample maven project) using maven job for slave2

- Repeat same steps as slave1 to create node "slave2".
- Create a job for sample maven project using maven job type.
- Provide description, enable the checkbox of restrict where this project can be run and enter the label "slave2".
- Under source code management, select git & give repository url of the maven project.
- Specify the branch_name, add build step --> execute shell & add required commands to generate pipeline script, apply & save then click on build now.
- Verify in slave2 instance by navigating to "f2" directory and run "ls" & you should find "workspace" then get inside workspace and verify your project files are present.

Step 7: Create job (sample java project) using declarative pipeline job for slave3

- Repeat same steps as slave1 to create node "slave3".
- Create a job for sample java project using declarative pipeline job type
- Provide syntax by replacing agent any with agent { label 'slave3' }, apply & save then click on build now.
- Verify in slave3 instance by navigating to "f3" directory and run "ls" & you should find "workspace" then get inside workspace and verify your project files are present.

Step 8: Create job (sample java project) using scripted pipeline job for slave4

- Repeat same steps as slave1 to create node "slave4".
- Create a job for sample java project using scripted pipeline job type
- Provide syntax in the script that is node('slave4'), apply & save then click on build now.
- Verify in slave4 instance by navigating to "f4" directory and run "ls" & you should find "workspace" then get inside workspace and verify your project files are present.

Sample for declarative pipeline job

```
pipeline {
  agent { label 'slave3' }
  stages {
    stage('Git-Clone') {
      steps {
        git 'https://github.com/TANVIP-6M/javaf.git'
      }
    }
    stage('Java Project') {
      steps {
        sh '''javac Test.java
            java Test.java'''
      }
    }
  }
}
```

Sample for scripted pipeline job

```
node('slave4') {
  stage('Git-Clone') {
    git 'https://github.com/TANVIP-6M/mavenf.git'
  }
  stage('Java Project') {
    sh '''javac Test.java
        java Test.java'''
  }
}
```

CI-CD implementation

1. Set up all four servers by completing every required pre-requisite before starting the integration process.

- Instance 1 --> Jenkins(8080)
- Instance 2 --> Sonarqube(9000)
- Instance 3 --> Nexus(8081)
- Instance 4 --> Tomcat(8080)
- Other port --> https, http, ssh

- Update
- Install java-17
- Install maven
- Install jenkins through link
- Start jenkins
- Enable port no 8080
- Configure jenkins

Jenkins

- Update
- Install java-17
- Install sonarqube through link
- Start sonarqube
- Enable port no 9000
- Sign in and configure sonarqube

Sonarqube

- Update
- Install java-17
- Install nexus through link
- Start nexus
- Enable port no 8081
- Sign in and configure nexus

Nexus

- Update
- Install java-17
- Install tomcat through link
- Start tomcat
- Enable port no 8080
- Sign in and configure tomcat

Tomcat

4. Maven project package

1. In jenkins create a new declarative pipeline job
2. Clone the spring petclinic github repository
3. Build the project to generate a petclinic.war package.

B. Sonarqube

1. Generate a token in sonarqube by clicking on administrator then go to security & create a new token by entering a name and click on generate.
2. Open manage jenkins and navigate to credentials.
 - Click on system (global).
 - Select add credentials.
 - In the kind dropdown choose secret text.
 - Paste the token in the secret text field.
 - In the id field enter the name you used while creating the token.
 - Provide a short description for the token and click on add credentials.
3. Install a plugin named "sonarqube scanner" in jenkins.
4. Go to manage jenkins and select configure system.
 - Under the sonarqube scanner section click add to set environment variables.
 - Give a name for the server for example sonarqube.
 - In the server URL field paste the URL of your sonarqube server.
 - Under credentials choose the token credentials you created earlier.
 - Click on apply and then save.
5. Open your CI-CD implementation pipeline job then go to configure and create a new stage for sonarqube.
 - In pipeline syntax search for the template named withSonarQubeEnv under sample steps.
6. Copy the generated syntax and paste it inside the new stage in your pipeline.
 - Modify the credentialsId and installation name as required then build the project to generate the package with a sonar report.

Ex :

```
stage('sonarqube') {  
    steps {  
        withSonarQubeEnv(installationName: 'sonarqube', credentialsId: 'jenkins-token') {  
            sh 'mvn verify sonar:sonar'  
        }  
    }  
}
```

C. Nexus

1. Go to manage jenkins then pluggins and install a plugin named "nexus artifact uploader".
2. Go to manage jenkins and navigate to credentials.
3. Add new credentials for nexus.
 - a) In the kind dropdown choose username with password.
 - b) Enter the username and password provide description and then click on create.
4. Open your CI-CD implementation pipeline job then go to configure and add a new stage for nexus.
5. In pipeline syntax select nexus artifact uploader as the sample step.
6. Fill in the necessary details such as version, artifactId, groupId provided in the petclinic pom.xml file from the github repository.
7. Under the nexus version field choose nexus3.
8. In the nexus URL specify the public IP address followed by port 8081.
Ex : <private-IP>:8081
9. Enter the repository name(Go to your nexus account --> repository --> spring-petclinic --> copy url(only repository name)) under the repository section.
10. Click on generate script.
11. Copy the generated script and paste it in the nexus pipeline stage.

Ex :

```
stage('nexus') {  
    steps {  
        nexusArtifactUploader artifacts: [[  
            artifactId: 'spring-framework-petclinic',  
            classifier: "",  
            file: 'target/petclinic.war',  
            type: 'war']],  
            credentialsId: 'nexus',  
            groupId: 'org.springframework.samples',  
            nexusUrl: '144.57.35.72:8081',  
            nexusVersion: 'nexus3',  
            protocol: 'http',  
            repository: 'spring-petclinic',  
            version: '6.2.8'  
        ]  
    }  
}
```

D. Tomcat

1. Go to jenkins and install a plugin named "ssh-agent"
2. Go to manage jenkins and add credentials specifically for tomcat.
3. In the kind dropdown, select "ssh username with private key".
4. Enter an id and description, and type ubuntu in the username field.
5. Paste the private key content and click on create to generate credentials for tomcat.
6. Open your CI/CD pipeline job and add one more stage for tomcat deployment.
7. In the pipeline syntax select a sample step as "ssh agent".
8. Make sure credentials are picked automatically then copy the syntax and paste it inside the tomcat stage.
9. Add the scp command to move the petclinic.war file from jenkins to the tomcat webserver.

Ex :

```
stage('tomcat') {  
    steps {  
        sshagent(['tomcat']) {  
            sh 'scp -o StrictHostKeyChecking=no target/petclinic.war ubuntu@19.123.45.199:/home/ubuntu/apache-  
tomcat-10.1.43/webapps'  
        }  
    }  
}
```

10. Run the build
11. Click on save and then click on build now to make sure the build completes successfully.
12. Access the application
 - Open the tomcat public URL in a browser and append /petclinic at the end to check whether the petclinic application is running --> `http://<tomcat_public_IP>:8080/petclinic`